

GMapping-SLAM Mapping

GMapping-SLAM Mapping

1. Course Content
2. GMapping Algorithm Introduction
3. Start the Agent
4. Run the Example
 - 4.1 Mapping Process
 - 4.2 Save the Map
5. Node Analysis
 - 5.1 Display Node Computation Graph
 - 5.2 TF Transformation
 - 5.3 GMapping Node Details
 - 5.4 Configuration File

1. Course Content

Basic: Run the sample program to perform SLAM mapping using the GMapping algorithm and save the grid map.

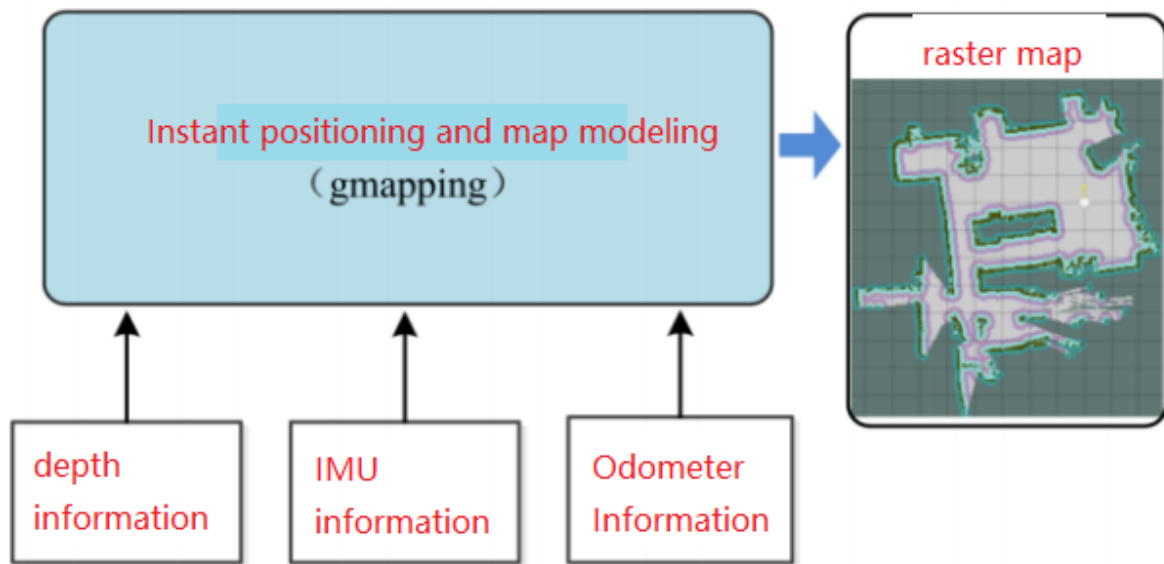
Advanced: Understand the configuration parameters of the GMapping algorithm.

2. GMapping Algorithm Introduction

- GMapping is only suitable for 2D laser scans with fewer than 1440 points per frame. If the number of points exceeds 1440, an issue like `[mapping-4] process has died` will occur.
- GMapping is a popular open-source SLAM algorithm based on the filter SLAM framework.
- GMapping is based on the RBPF (Rao-Blackwellized Particle Filter) algorithm, where the localization and mapping processes are separated. Localization is performed first, followed by mapping.
- GMapping makes two main improvements to the RBPF algorithm: an improved proposal distribution and selective resampling.

Advantages: GMapping can build indoor maps in real-time. It requires less computational power and offers higher accuracy for small-scale mapping.

Disadvantages: As the scene size increases, more particles are needed. Since each particle carries a map, both memory and computational requirements increase when building large maps. Therefore, it is not suitable for large-scale mapping. It also lacks loop closure detection, which can cause map misalignment when a loop is closed. Although increasing the number of particles can help close the map, it comes at the cost of increased computation and memory.



3. Start the Agent

Note: If it is already running, you do not need to start it again.

Enter the command in the vehicle's terminal:

```
start_agent
```

The terminal will print the following information, indicating a successful connection:

```

mircoROS_Agent
[1764932423.553407] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x000(6), subscri
ber_id: 0x000(4)
[1764932423.557165] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x006(2), participant_
id: 0x000(1)
[1764932423.560435] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1764932423.565261] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1764932423.569631] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x14F22590, topic_id: 0x007(2), participant_
id: 0x000(1)
[1764932423.573464] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x14F22590, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1764932423.578415] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x14F22590, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)

```

4. Run the Example

4.1 Mapping Process

- Note: When mapping, a slower speed (especially rotational speed) yields better results. If the speed is too high, the mapping quality will be poor.
- Launch the mapping command in the terminal:

For Raspberry Pi 5 users, if the ROS container is not started, you need to start it first,

```
bringup_m3
```

Open a terminal inside the container,

```
exec_m3
```

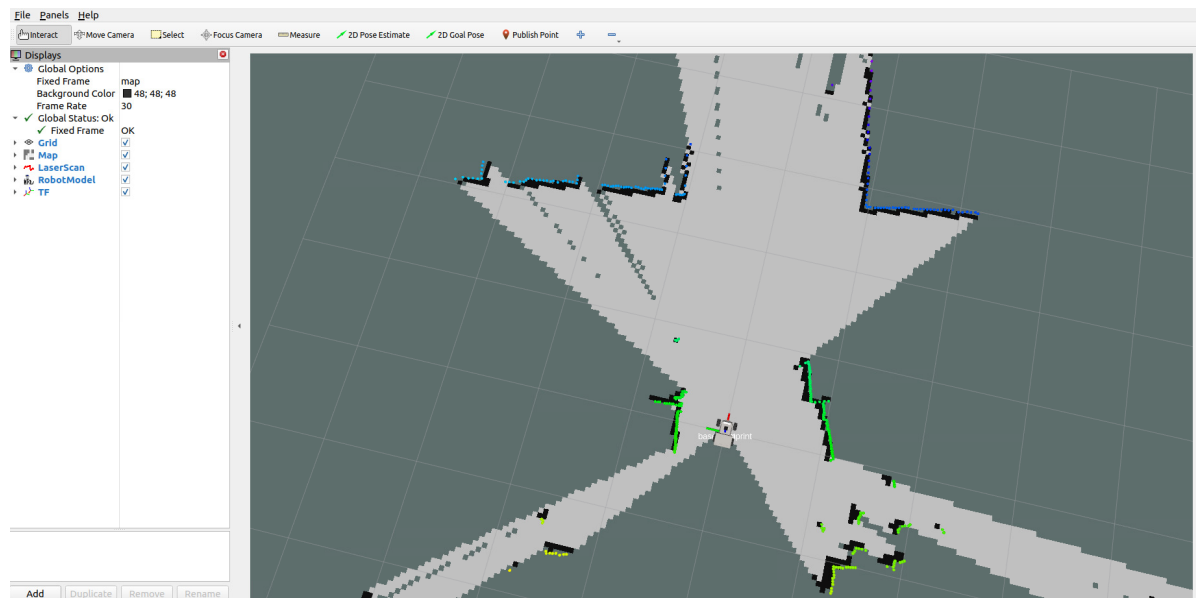
Enter the command,

(ORIN & PI5 boards)

```
ros2 launch m3_bringup gmapping.launch.py
```

(RDK board) Run in separate steps, with visualization launched on the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`,

```
# On the virtual machine
rviz2 -d
/home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/gmapping_rviz.rviz
# On the host machine
ros2 launch m3_bringup gmapping.launch.py gui:=False
```



- To control the vehicle's movement, you can use either keyboard or joystick control. Here, we use keyboard control as an example:

```
keyboard_control
```

- Click on the terminal window with the mouse and press `z` to reduce the speed appropriately.

```

yahboom@yahboom-virtual-machine: ~
yahboom@yahboom-virtual-machine: ~ 126x24

For Holonomic mode (strafing), hold down the shift key:
-----
  U    I    O
  J    K    L
  M    <    >

q : up (+z)
o : down (-z)

Anything else : stop

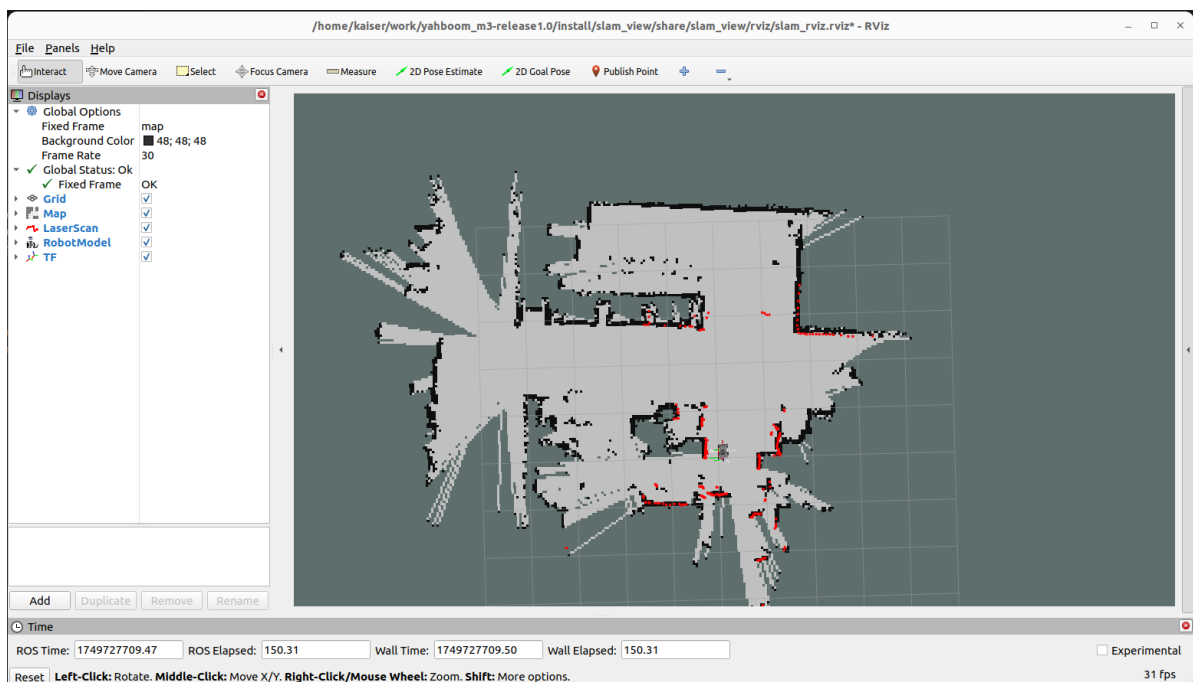
q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:    speed 0.5      turn 1.0
currently:    speed 0.45     turn 0.9
currently:    speed 0.405    turn 0.81
currently:    speed 0.3645000000000005    turn 0.7290000000000001
currently:    speed 0.3280500000000006    turn 0.6561000000000001

```

Press `I`, `<`, `J`, `L` to control the car to move forward, backward, turn left, and turn right, respectively, to complete the mapping.



4.2 Save the Map

- Run the following command in the terminal to save the grid map:

```
ros2 launch m3_bringup save_map.launch.py
```

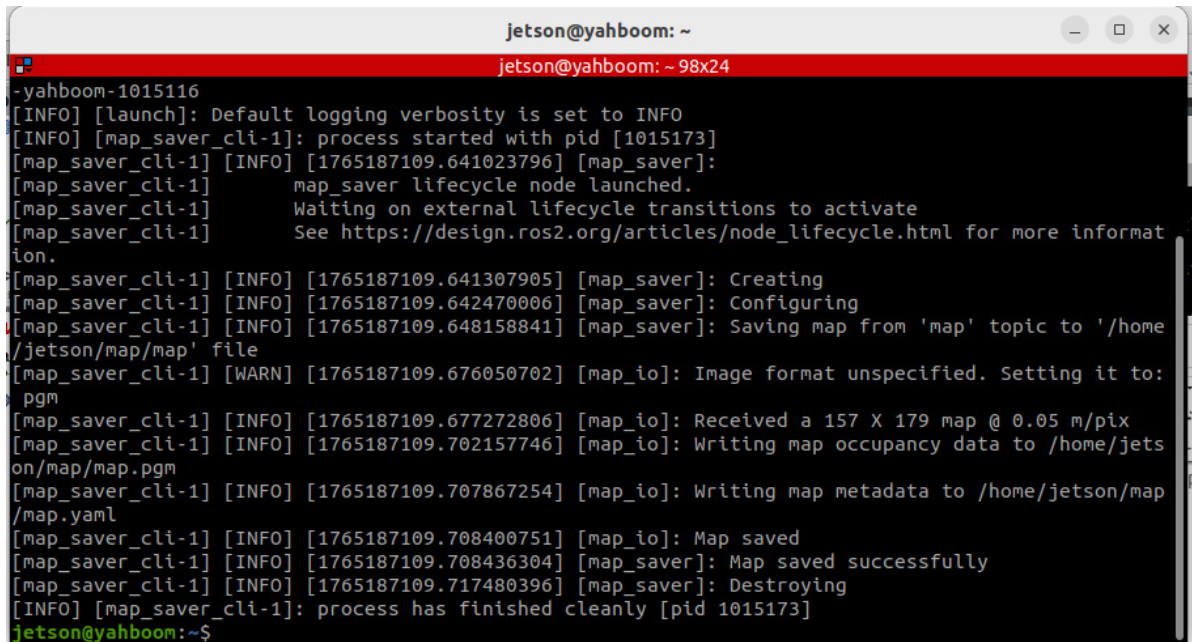
[!TIP]

Optional Parameters:

`map_name`: Specifies the name of the saved map, with the default value being `map`.

For example, to save a map named `test.yaml`, add the parameter: `ros2 launch m3_bringup save_map.launch.py map_name:=test`

The terminal prompt **Map saved successfully** indicates that the map was saved successfully. If it fails, try saving again.

A terminal window titled 'jetson@yahboom: ~' with a red title bar. The terminal shows the output of a ROS2 launch command. It starts with '[INFO] [launch]: Default logging verbosity is set to INFO'. Then, '[map_saver_cli-1] [INFO] [1765187109.641023796] [map_saver]: map_saver lifecycle node launched.' followed by 'Waiting on external lifecycle transitions to activate' and a link to a ROS2 article. Then, '[map_saver_cli-1] [INFO] [1765187109.641307905] [map_saver]: Creating' and '[map_saver_cli-1] [INFO] [1765187109.642470006] [map_saver]: Configuring'. A red line indicates an error: '[map_saver_cli-1] [INFO] [1765187109.648158841] [map_saver]: Saving map from 'map' topic to '/home/jetson/map/map' file'. This is followed by a warning: '[map_saver_cli-1] [WARN] [1765187109.676050702] [map_io]: Image format unspecified. Setting it to: pgm'. Then, '[map_saver_cli-1] [INFO] [1765187109.677272806] [map_io]: Received a 157 X 179 map @ 0.05 m/pix', '[map_saver_cli-1] [INFO] [1765187109.702157746] [map_io]: Writing map occupancy data to /home/jetson/map/map.pgm', and '[map_saver_cli-1] [INFO] [1765187109.707867254] [map_io]: Writing map metadata to /home/jetson/map/map.yaml'. The final successful messages are '[map_saver_cli-1] [INFO] [1765187109.708400751] [map_io]: Map saved', '[map_saver_cli-1] [INFO] [1765187109.708436304] [map_saver]: Map saved successfully', and '[map_saver_cli-1] [INFO] [1765187109.717480396] [map_saver]: Destroying'. The process ends with '[INFO] [map_saver_cli-1]: process has finished cleanly [pid 1015173]' and the prompt 'jetson@yahboom:~\$'.

The default map save path is as follows:

```
~/map/map.yaml
```

This includes a PGM image and a YAML file, `yahboom_map.yaml`.

```
image: yahboom_map.pgm
mode: trinary
resolution: 0.05
origin: [-10, -10, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

Parameter analysis:

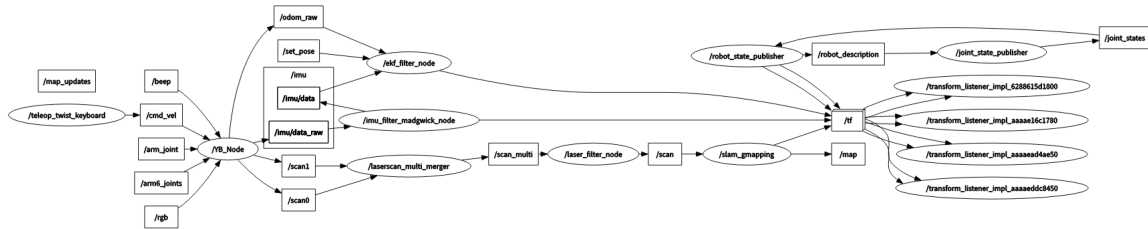
- `image`: The path to the map file, which can be an absolute or relative path.
- `mode`: This attribute can be `trinary`, `scale`, or `raw`, depending on the chosen mode. `trinary` is the default.
- `resolution`: The resolution of the map, in meters/pixel.
- `origin`: The 2D pose (x, y, yaw) of the bottom-left corner of the map. Here, `yaw` is a counter-clockwise rotation (yaw=0 means no rotation). Many parts of the system currently ignore the yaw value.
- `negate`: Whether to invert the meaning of white/black and free/occupied (the interpretation of thresholds is not affected).
- `occupied_thresh`: Pixels with an occupancy probability greater than this threshold are considered fully occupied.
- `free_thresh`: Pixels with an occupancy probability less than this threshold are considered fully free.

5. Node Analysis

5.1 Display Node Computation Graph

Run in the virtual machine terminal:

```
ros2 run rqt_graph rqt_graph
```

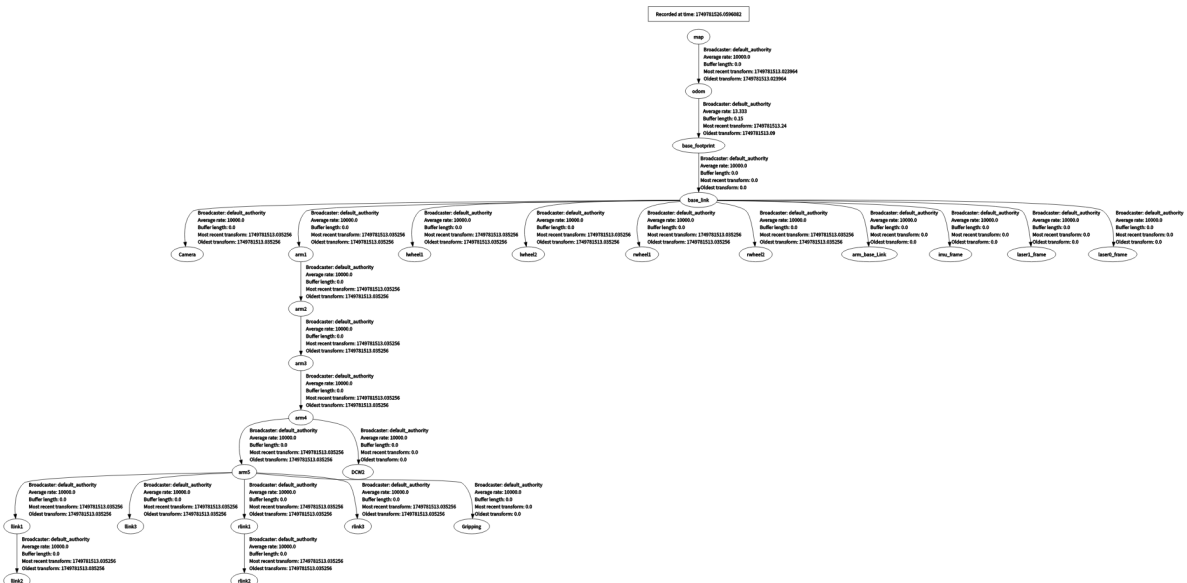


5.2 TF Transformation

Run in the virtual machine terminal:

```
ros2 run rqt_tf_tree rqt_tf_tree
```

The image size is too large. The original image can be viewed in this lesson's course folder.



5.3 GMapping Node Details

```
ros2 node info /slam_gmapping
```

Entering the above command in the terminal allows you to view the topics subscribed to and published by the GMapping node.

```
/slam_gmapping
subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
```

```

    /scan: sensor_msgs/msg/LaserScan
Publishers:
    /entropy: std_msgs/msg/Float64
    /map: nav_msgs/msg/OccupancyGrid
    /map_metadata: nav_msgs/msg/MapMetaData
    /parameter_events: rcl_interfaces/msg/ParameterEvent
    /rosout: rcl_interfaces/msg/Log
    /tf: tf2_msgs/msg/TFMessage
Service Servers:
    /slam_gmapping/describe_parameters: rcl_interfaces/srv/DescribeParameters
    /slam_gmapping/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
    /slam_gmapping/get_parameters: rcl_interfaces/srv/GetParameters
    /slam_gmapping/list_parameters: rcl_interfaces/srv/ListParameters
    /slam_gmapping/set_parameters: rcl_interfaces/srv/SetParameters
    /slam_gmapping/set_parameters_atomically:
rcl_interfaces/srv/SetParametersAtomically
Service Clients:

Action Servers:

Action Clients:

```

5.4 Configuration File

- Configuration file path:

```
~/yahboomM3_ws/user_ws/src/m3_bringup/config/slam_gmapping.yaml
```

- Default configuration parameters:

```

slam_gmapping:
  ros__parameters:
    # Angular update threshold (radians): Update the map when the robot rotates
    more than this angle
    angularUpdate: 0.25
    # Angular sampling step (radians): Angular search step for scan matching
    astep: 0.05
    # Robot base coordinate frame name
    base_frame: base_footprint
    # Map coordinate frame name
    map_frame: map
    # Odometry coordinate frame name
    odom_frame: odom
    # Iteration termination condition for scan matching: Stop iterating when
    parameter change is less than this value
    delta: 0.05
    # Maximum number of iterations for scan matching
    iterations: 5
    # Kernel size for scan matching, affects matching smoothness
    kernelSize: 1

    # Distance sampling range for large-scale scan matching (meters)
    lasamplerange: 0.005
    # Distance sampling step for large-scale scan matching (meters)

```



```

lasamplestep: 0.005
# Linear update threshold (meters): Update the map when the robot moves more
than this distance
linearUpdate: 0.5
# Distance sampling range for small-scale scan matching (meters)
llsamplerange: 0.01
# Distance sampling step for small-scale scan matching (meters)
llsamplestep: 0.01

# Measurement noise covariance for laser scan
lsigma: 0.075
# Number of laser beams to skip: 0 means use all laser points
lskip: 0
# Linear sampling step (meters): Linear search step for scan matching
lstep: 0.05
# Map update time interval (seconds)
map_update_interval: 3.0
# Maximum laser range (meters): Points beyond this distance are ignored
maxRange: 6.0
# Maximum usable laser range (meters): Maximum distance used for mapping
maxUrange: 4.0
# Minimum score threshold for scan matching: Matches below this score are
discarded
minimum_score: 0.0
# Occupancy probability threshold: Cells with a probability greater than this
are considered occupied
occ_thresh: 0.25
# Gain factor: Affects the speed of map updates
ogain: 3.0
# Number of particles for the particle filter
particles: 30
# QoS (Quality of Service) parameter settings
qos_overrides:
  /parameter_events:
    publisher:
      depth: 1000          # Message queue depth
      durability: volatile # Durability policy
      history: keep_all    # History policy
      reliability: reliable # Reliability policy
  /tf:
    publisher:
      depth: 1000
      durability: volatile
      history: keep_last   # Keep only the latest message
      reliability: reliable

# Resampling threshold: Trigger resampling when the number of effective
particles is below this ratio
resampleThreshold: 0.5
# Angular noise covariance for scan matching
sigma: 0.05
# Odometry model parameter: Proportional linear error from linear motion
srr: 0.1
# Odometry model parameter: Proportional angular error from linear motion
srt: 0.2
# Odometry model parameter: Proportional linear error from angular motion
str: 0.1
# Odometry model parameter: Proportional angular error from angular motion
stt: 0.2

```



```
# Time update interval (seconds): Update the map even if the robot has not
moved
temporalUpdate: 1.0
# Coordinate transform publication period (seconds)
transform_publish_period: 0.05
# Whether to use simulation time
use_sim_time: false
# Maximum x-range of the map (meters)
xmax: 100.0
# Minimum x-range of the map (meters)
xmin: -100.0
# Maximum y-range of the map (meters)
ymax: 100.0
# Minimum y-range of the map (meters)
ymin: -100.0
```

These are the configurable parameters for GMapping. If you need to modify the configuration parameters, you must recompile the `slam_gmapping` package in the `M3Pro_ws` workspace for the changes to take effect:

```
colcon build --packages-select slam_gmapping
```